Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 257 (2025) 561–568

Procedia Computer Science

www.elsevier.com/locate/procedia

The 16th International Conference on Ambient Systems, Networks and Technologies (ANT)
April 22-24, 2025, Patras, Greece

Webcam Eye-Tracking Browser Extension For General Navigation

Quinth Anthony A. Razuman, Brixzel John Q. Mabala, Malikey M. Maulana*

Mindanao State University - Iligan Institute of Technology, Iligan City 9200, Philippines

Abstract

Webcam eye-tracking has emerged as a cost-effective alternative to traditional eye-tracking technologies, offering a promising solution for individuals with disabilities who face challenges in using standard input devices. In this study, the researchers developed a web browser extension leveraging webcam eye-tracking for web browsing and navigation. The extension was created using Webgazer.js, a webcam eye-tracking JavaScript library, and complemented by the development of four web applications optimized for webcam eye-tracking. Through user evaluations and testing, the system demonstrated encouraging results. Users found the system to be both useful and user-friendly, as evidenced by a progressive decline in error rates over time. This trend indicated successful iterative system improvements and increased user proficiency. Although variations in error rates were observed across different applications, suggesting varying levels of intuitiveness and user-friendliness, the overall findings affirmed the system's utility and usability. This research achieved its objectives by effectively showcasing the extension's potential as a robust tool for individuals with hand disabilities.

Keywords: Eye-tracking; Webcam eye-tracking; webgazer.js; web browser; browser extension; accessibility;

1. Introduction

Eye-tracking is a sensor technology that detects presence and tracks gaze in real time, converting eye movements into a data stream containing pupil position, gaze vector, and gaze point. Two primary technologies enable eye-tracking: premium eye trackers, which use infrared to illuminate the iris, and standard webcams, which acquire eye-tracking data without such enhancements. While premium eye trackers are highly accurate, webcams offer an affordable alternative but lack the precision, sensitivity, and robustness of high-end equipment. Webcam eye-tracking requires stable user positioning and is affected by webcam quality, especially in low light or with low frame rates and resolutions.

* Corresponding author. Tel.: +63-915-267-1799
E-mail address: malikey.maulana@g.msuiit.edu.ph

1877-0509
10.1016/j.procs.2025.03.072

Webcam eye-tracking research has advanced through tools like WebGazer [1], a JavaScript library developed at Brown University. WebGazer enables real-time inference of eye-gaze locations on web pages using a webcam. It self-calibrates based on user interactions, mapping eye characteristics to screen positions. However, WebGazer remains largely experimental and lacks implementation in practical applications.

Szymon Deja has developed several applications leveraging webcam eye-tracking, including GazePointer, GazeRecorder, GazeCloud, GazeScroll, GazeBoard, and GazeFlow [2]. Each serves a unique function: GazePointer moves the mouse cursor using gaze, GazeRecorder captures eye-tracking data and screen content, GazeCloud generates gaze heatmaps, GazeScroll automates scrolling, and GazeBoard facilitates text entry. These tools are built on GazeFlow, an engine designed for gaze-based heatmap research, with applications in website usability, film screening, advertising, and accessibility. However, these applications operate independently and lack integration into a unified platform.

Two studies explore browser-based eye-tracking. GazeTheWeb [4] enables gaze-based browsing, supporting features like bookmarking and scrolling. It offers improved usability over emulation methods like OptiKey [3] but is compatible only with high-end eye trackers such as SMI RED-n, Tobii EyeX, and Tobii 4C, excluding webcams. Similarly, a 2020 study introduced a gaze-based browser [5] featuring hands-free navigation with five link-selection and page-scrolling techniques. The study highlights challenges in link selection and emphasizes the role of large buttons and intuitive graphical elements in enhancing usability. Both works focus on premium eye trackers, leaving a gap in gaze control solutions for webcams.

Eye-tracking technology has transformative potential for persons with disabilities (PWD) by enabling hands-free interaction with digital systems, advancing accessibility in line with the United Nations Sustainable Development Goals (SDGs). Goal 10 emphasizes reducing inequalities, and affordable webcam-based eye-tracking aligns with this aim by fostering inclusive digital environments [6, 7].

This study addresses the gap by presenting a fully functional web browser extension for webcam-based eye-tracking. It allows users to browse the internet using gaze control, offering a cost-effective and accessible solution to enhance digital participation for PWD.

2. Materials and Method

The system developed is a browser extension that uses webcam eye-tracking as an alternative input method for web navigation. It includes features such as gaze-triggered clicking, gaze-activated bookmarks, a speed setting for the gaze-tracking circle, and a calibration function. These features offer a new, user-friendly way to navigate the web and interact with applications, particularly for individuals with hand disabilities. To demonstrate the system's practical applications, four web applications have been developed, including a game, a communication tool, a food ordering system, and a YouTube video browser.

2.1. System flow diagram

As shown in Fig. 1, the system flow diagram outlines the sequence of operations from browser launch to termination. When the user opens the browser, the system first checks if the eye-tracking extension is enabled. If not, the user is prompted to activate it. Once enabled, the system verifies whether the browser has been calibrated for eye-tracking; if calibration is incomplete, the user must perform it before proceeding.

After activation and calibration, webcam eye-tracking becomes active and an interface overlay appears, allowing continuous monitoring of the user's gaze. If the user's gaze remains fixed on a link for more than four seconds, the system triggers the click function and automatically redirects to the corresponding page. Similarly, if the gaze is maintained on a button for over four seconds, the associated function is executed. This process continues until the user disables the extension or closes the browser, at which point the system flow ends.

2.2. Design of gaze indicator and click feature

The default gaze indicator in WebGazer.js proved too rapid for smooth navigation. As shown in Fig. 2, we designed a custom blue circle that updates every 0.5 seconds with restricted motion to improve control. This indicator freezes when the user is absent and resumes when they return.

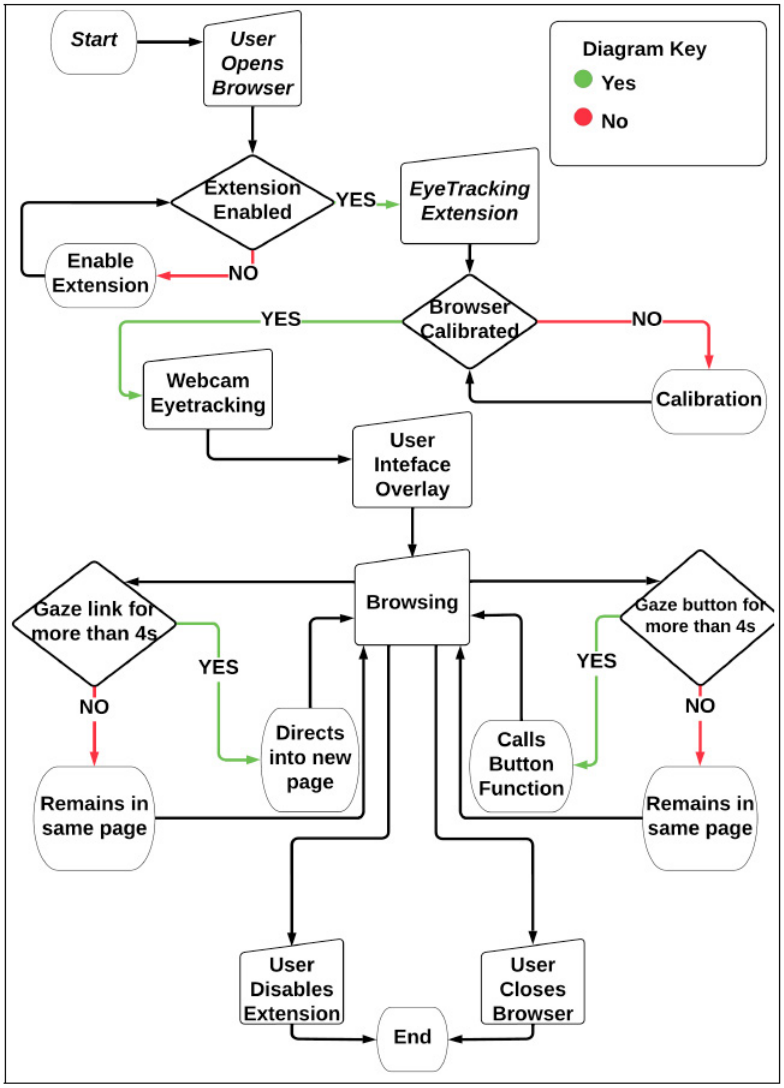


Fig. 1: System flow diagram

Since WebGazer.js does not provide a built-in click function, we implemented a gaze-triggered click feature. A fixation of 4 seconds on an element triggers a click with visual feedback. To minimize misclicks, this feature is initially disabled and activates 3 seconds after the page loads.

2.3. Design of the calibration

We implemented a 9-point calibration overlay (see Fig. 3) to guide users. Following a 10-second instructional countdown, users fixate on sequentially illuminated circles (red turning to green) for 5 seconds each. Calibration data is then stored in Chrome Local Storage.

2.4. Design of the browser extension popup and bookmark feature

The browser extension interface unifies a control popup and a bookmark overlay. As shown in Fig. 4a, the popup provides buttons to start calibration, clear data, toggle the overlay, and adjust the gaze indicator speed. Additionally,

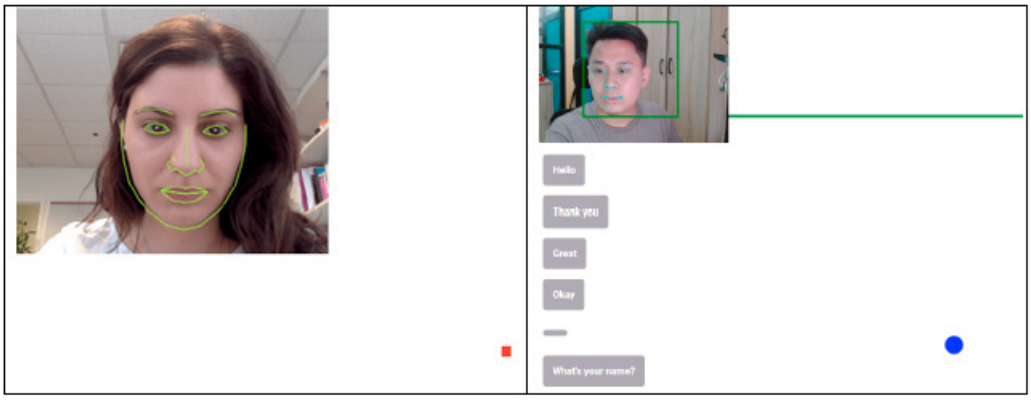
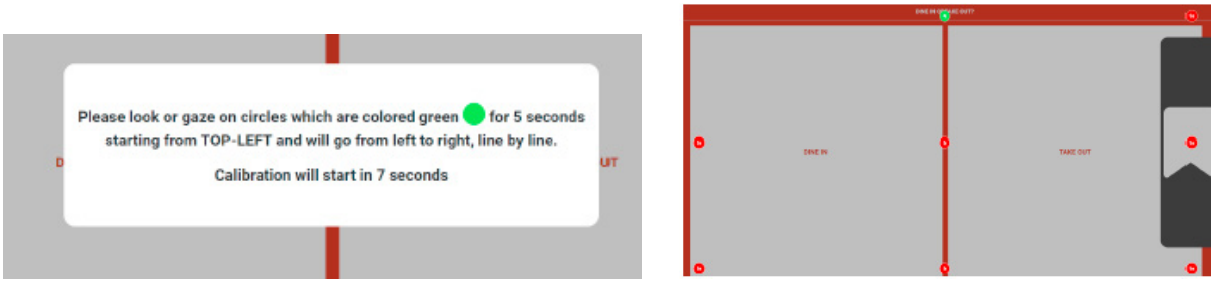


Fig. 2: (a) built-in gaze indicator; (b) redesigned gaze indicator.

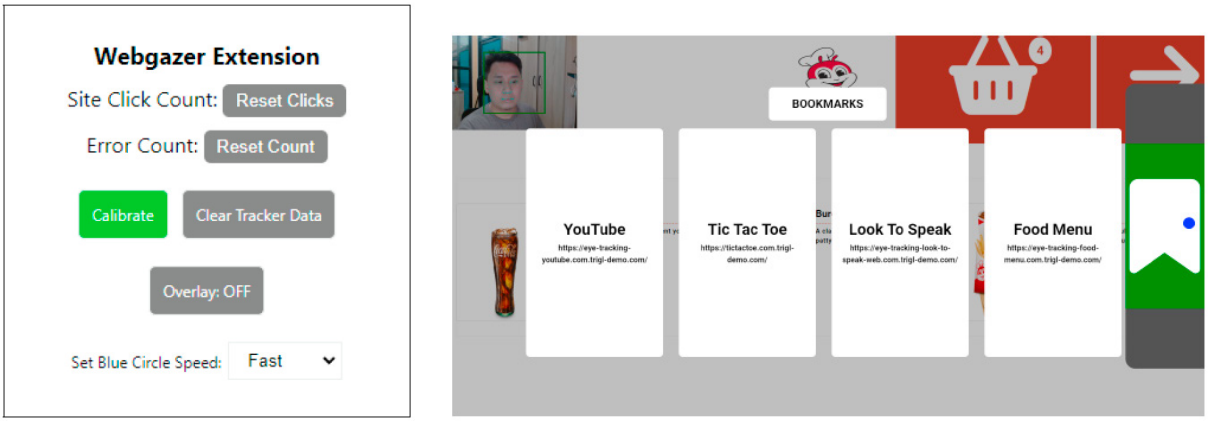


(a) calibration instruction

(b) calibration process

Fig. 3: Calibration

a bookmark overlay (see Fig. 4b) offers quick access to web applications by displaying four columns of links when activated. This feature is user-toggleable and, by default, is disabled for most applications (except the Food Menu app) to minimize interference during regular browsing. User feedback was continually integrated to optimize usability and accessibility.



(a) extension popup

(b) bookmark of the extension

Fig. 4: browser extension

2.5. Web applications

The researchers developed several web applications compatible with the eye-tracking browser extension. A simple tic-tac-toe game demonstrated the system's viability, while another application—based on the Look to Speak mobile app [8]—displays words in two large boxes, allowing users to select phrases with their eyes; the chosen phrases are then vocalized by an AI voice generator. Additionally, a food-ordering prototype presents a restaurant menu with a cart and checkout page, where the final receipt is read aloud using AI, and a YouTube-like application, featuring a tailored interface with oversized buttons, leverages video.js and YouTube's API for data retrieval.

The general design of these web applications emphasizes usability and aims to minimize navigation errors. To reduce misclicks, link elements and buttons are sized with a minimum height of 300px (or at least one-quarter of the screen height) and a minimum width of 300px. Furthermore, these interactive elements are primarily placed on the left side of the screen—especially in the top-left corner—to align with the typical F-shaped scanning pattern observed in previous studies [9].

Additional design considerations include minimizing overlapping elements that can hinder navigation; for example, providing an option to hide obstructive overlays improves the user experience. Finally, pagination is favored over scrolling, as testers encountered difficulties with scrolling. Limiting content to fit within screen dimensions further simplifies navigation and enhances accessibility.

2.6. Evaluation

This section evaluates the effectiveness of the webcam eye-tracking system in facilitating interaction with the developed web applications.

2.6.1. Respondents

The study involved four respondents who tested the system using a 1080p webcam and a 1080p resolution monitor. Eyeglasses were not permitted during testing to ensure accurate eye-tracking results. Although the sample size is small, usability research suggests that extensive testing is not always necessary to identify key usability issues. According to Nielsen [10], testing with five users can uncover approximately 85% of usability problems, making small usability tests both practical and effective.

2.6.2. Method of gathering data

The data collection process began with respondents undergoing a calibration phase to configure the eye-tracking system accurately. Respondents then tested four web applications, and their actions were recorded using the browser extension's eye-tracking capabilities. After the tests, participants completed surveys to provide feedback. Improvements to the applications were implemented based on these insights, and the same respondents were invited to test updated versions. This iterative process ensured continuous refinement of the system.

The evaluation framework consisted of three sections:

1. Metrics: Quantitative data, including the number of user moves, selections, and errors for each application, were recorded. Respondents also rated their experience using the webcam eye-tracking system and its effectiveness in assisting individuals with hand disabilities.
2. Technology Acceptance Model (TAM): Respondents assessed the system's perceived usefulness, ease of use, and flexibility for people with hand disabilities. Descriptive statistics such as means and ranges summarized trends, while a case study approach analyzed individual responses for unique insights [11].
3. System Usability Scale (SUS): Usability was evaluated based on factors such as ease of use, functionality integration, and learnability. Individual SUS scores were calculated, and patterns or notable findings were highlighted [12].

2.6.3. Data analysis

Given the small sample size, a descriptive and case study approach was adopted [13]. Usability research supports the effectiveness of small-scale testing. Nielsen [10] argues that testing with just five users can reveal the majority of usability issues, making iterative testing with small groups a valid approach.

3. Results and Discussions

3.1. Browser extension

The Webcam Eye-Tracking Browser Extension yielded several key findings. Gaze clicking was implemented to enable hands-free browsing by translating eye movements into input, with accuracy and responsiveness as critical factors. A calibration process allowed users to optimize performance for their environment, while an intuitive overlay facilitated efficient navigation between web applications. Additional user customizations, such as adjusting gaze circle speed and toggling bookmark overlays, further enhanced usability.

Furthermore, treating calibration data as global reduced recalibration frequency, enabling seamless website navigation. Recommendations for improving accuracy included positioning the eyes closer to the webcam, maintaining a steady posture during calibration, and ensuring a distraction-free environment. However, the Webgazer.js library's inability to detect blinks or eye closures resulted in occasional inaccuracies, highlighting the need for future research to integrate advanced facial recognition capabilities.

3.2. Evaluation of the web applications

The evaluation of web applications was conducted by analyzing error percentages across four applications—Tic Tac Toe, Look To Speak, Food Menu, and YouTube—tested by four users over three rounds. As shown in Fig. 5, results demonstrated a consistent decline in error rates as users became more familiar with the system. User 1 showed the most significant improvement, reducing their error rate from 50% in the first round on YouTube to 5.40% in the third. User 2 exhibited a similar trend, with error rates decreasing from 42.78% to 16.91%. User 3 reduced their error rate dramatically, from 48.96% to just 3.13%, while User 4, despite having the highest overall error percentage (22.26%), improved from 30.83% to 1.67%.

Overall, the average error percentage across all users dropped from 38.28% in the first round to 15.89% in the second and further to 6.78% in the third. This steady improvement highlights increased user proficiency and the positive impact of iterative enhancements to the browser extension and web applications. The results underscore that familiarization and system updates contribute to significant reductions in user errors, enhancing the usability and effectiveness of the applications.

3.3. Technology Acceptance Model (TAM) evaluation

The Technology Acceptance Model (TAM) was employed to evaluate user attitudes toward the tested applications, focusing on perceived usefulness and perceived ease of use. Participants rated these factors on a scale from 1 to 5, where 1 indicated least useful or least easy to use, and 5 indicated most useful or most easy to use. The results, summarized in Table 1, revealed that users generally found the applications both useful and easy to use, with average scores of 4.83 for perceived usefulness and 4.17 for perceived ease of use. These high scores indicate a positive user perception of the system. However, it is important to note that these scores are subjective and may be influenced by individual user preferences. Additionally, the small sample size limits the generalizability of these findings. Future studies with a larger sample size would help provide more reliable data.

Table 1: TAM Data Results

	Perceived Usefulness	Ease of Use of the System
User 1	4.33	3.67
User 2	5.00	4.00
User 3	5.00	4.00
User 4	5.00	5.00
Average	4.83	4.17

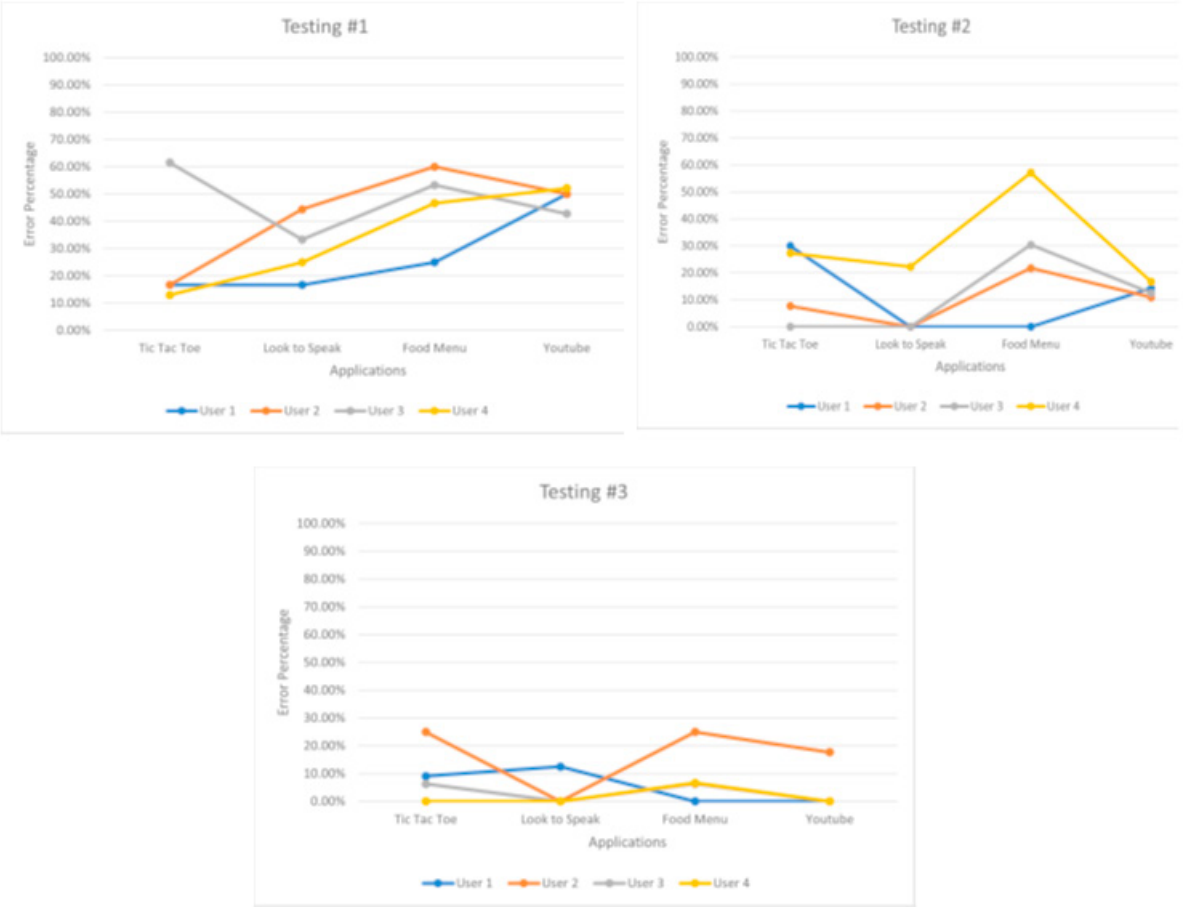


Fig. 5: error percentage results

3.4. System Usability Scale (SUS) Results

The System Usability Scale (SUS) was utilized to assess the user interface and overall usability of the system. As presented in Table 2, the SUS scores reflect the overall perceived usability, with higher scores indicating better usability. The average SUS score across all users was 71.88, which falls within the ”Good” usability range, indicating that, on average, users found the system to meet acceptable usability standards. Notably, User 3 provided an ”Excellent” rating, suggesting that this user found the system highly user-friendly and efficient. However, Users 1 and 2, while providing scores within acceptable ranges, had lower scores, highlighting areas for potential improvement. These scores suggest the need for further investigation into their feedback to identify specific usability issues. Overall, the SUS results align with the high Perceived Usefulness and Ease of Use scores from the TAM evaluation, reinforcing the system’s effectiveness and user-friendliness.

The test results indicate that the web applications were generally considered useful and relatively easy to use by participants. The observed decrease in error percentages across testing rounds suggests that the system improvements made after each round were effective, and that users became more familiar and efficient with the applications over time. However, the variation in error percentages between different applications implies that some applications may be more intuitive and user-friendly than others. The high ratings in the TAM questionnaire and the good average SUS score further support the positive reception of the web applications. However, the range of SUS scores and the differences in error percentages among users suggest that individual factors, such as technology familiarity and learning speed, may influence usability. Given the small sample size, these results should be considered indicative

Table 2: SUS Data Results

	SUS Score	SUS Score Interpretation
User 1	67.5	OK
User 2	60	OK
User 3	90	EXCELLENT
User 4	70	OK
Average	71.88	GOOD

rather than definitive. Further testing with a larger user group is recommended to confirm these findings and explore individual differences more thoroughly.

4. Conclusion and Future Work

This study developed an accessible webcam eye-tracking system for web navigation, particularly aiding individuals with hand disabilities. The system's iterative improvements—such as gaze-triggered clicking, a streamlined calibration process, and an intuitive interface—demonstrate its potential and effectiveness.

Although only four users were involved, Nielsen [10] shows that testing with five users can reveal around 85% of usability issues, supporting our small-scale, iterative approach. Future research should involve a larger, more diverse sample to further validate these findings.

Enhancements will focus on improving calibration (e.g., using a 13-point method), refining the interface, and increasing customization. Additionally, addressing limitations in WebGazer.js, such as its inability to detect blinks, will further enhance accuracy and usability. Expanding the system's compatibility with more web applications is also recommended.

Overall, this work lays a strong foundation for further development of webcam-based eye-tracking systems, underscoring their potential to significantly improve digital accessibility for users with disabilities.

References

[1] A. Papoutsaki, P. Sangkloy, J. Laskey, N. Daskalova, J. Huang, and J. Hays, "WebGazer: Scalable webcam eye tracking using user interactions," in *Proceedings of the 25th International Joint Conference on Artificial Intelligence (IJCAI)*, 2016, pp. 3839–3845, AAAI.

[2] Szymon Deja, "Online eye tracking: Webcam eye-tracking software", *GazeRecorder*, https://gazerecorder.com/, [Online; accessed 6-Dec-2022].

[3] "What is Optikey?", *Optikey*, http://www.optikey.org/help/what-is-optikey, [Online; accessed 8-Aug-2022].

[4] R. Menges, C. Kumar, D. Müller, and K. Sengupta, "Gazetheweb: A gaze-controlled web browser," in *Proceedings of the 14th International Web for All Conference*, 2017, pp. 1–2.

[5] M. Casarini, M. Porta, and P. Dondi, "A gaze-based web browser with multiple methods for link selection," in *ACM Symposium on Eye Tracking Research and Applications*, 2020, pp. 1–8.

[6] United Nations, "Disability and Development", n.d., https://www.un.org/development/desa/disabilities/

[7] United Nations, "Sustainable Development Goals", n.d., https://sdgs.un.org

[8] R. Cave, "Look to speak helps people communicate with their eyes," *Google*, Google, Dec. 2020. [Online]. Available: https://blog.google/outreach-initiatives/accessibility/look-to-speak/. [Accessed: Aug. 24, 2021].

[9] SproutReach, *F-shaped Reading Pattern on the Web: Revisiting its meaning 12 yrs after its introduction*, 2018. [Online]. Available: https://sproutreach.com/blog/f-shaped-reading-pattern-on-the-web-revisiting-its-meaning/. [Accessed: 14- Dec- 2024].

[10] Nielsen, J. (2000). *Why You Only Need to Test with 5 Users*. Nielsen Norman Group. Retrieved from https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/

[11] F. D. Davis, "Perceived usefulness, perceived ease of use, and user acceptance of information technology," *MIS Quarterly*, vol. 13, no. 3, pp. 319–340, 1989.

[12] J. Brooke, "SUS: a quick and dirty usability scale," in *Usability Evaluation in Industry*, Taylor & Francis, 1995, pp. 189–194.

[13] J. W. Creswell, *Qualitative Inquiry and Research Design: Choosing Among Five Approaches*, SAGE Publications, 2013.

[14] C. L. Aberson, *Applied Multivariate Statistics for the Social Sciences*, Routledge, 2019.